

Documento de Política de Manejo de Fallos

Fase 2 — Modos de Ejecución y Syscalls | Sistemas Operativos

1. Objetivo

Este documento define cómo el kernel de la Fase 2 detecta, clasifica y contiene los fallos en modo usuario (data aborts y prefetch aborts). El objetivo de diseño es el aislamiento de fallos: la tarea que falla es aislada y terminada mientras el kernel y el resto de las tareas pares continúan ejecutándose con normalidad.

2. Vectores de Excepción ARM Utilizados

Vector ARM	Handler (root.s)	Función C
Data Abort (0x10)	data_handler	data_abort_c(fault_addr)
Prefetch Abort (0x0C)	prefetch_handler	prefetch_abort_c(fault_addr)
Undefined Instruction	undefined_handler	Halt del kernel (b . — bucle infinito)

3. Tabla de Clasificación de Fallos (Clasificación → Acción)

La siguiente tabla define el mapeo clasificación → acción para todos los escenarios de fallo en esta fase.

Origen del fallo	Mecanismo ARM	Campo type	Acción
Fetch de instrucción ilegal (ej. 0xDEADC0DE)	Prefetch Abort	prefetch_abort	Tarea TERMINATED; planificador elige la siguiente tarea READY
Desreferencia de puntero nulo	Data Abort	data_abort	Tarea TERMINATED; planificador elige la siguiente tarea READY
Acceso a memoria fuera de región	Data Abort	data_abort	Tarea TERMINATED; planificador elige la siguiente tarea READY
Acceso desalineado (si MMU/MPU está configurada)	Data Abort	data_abort	Tarea TERMINATED; planificador elige la siguiente tarea READY
Instrucción indefinida	Undefined Instruction	(fallo del kernel)	Halt del kernel — b . (no es un fallo de usuario; se trata como fatal)
SYS_WRITE con puntero inválido	Validación en path de syscall	(error -3 de syscall)	Retorna -3; la tarea continúa (contenido dentro de la syscall)

4. Flujo de Contención de Fallos

Los siguientes pasos se ejecutan para cada fallo en modo usuario (data abort o prefetch abort):

1. La CPU toma la excepción de forma síncrona respecto a la instrucción que falla; ARM vectoriza a `data_handler` o `prefetch_handler` en modo abort.
2. El prólogo en ensamblador (`root.s`) ajusta LR (`sub lr, lr, #4`), guarda `r0–r12` y LR en el stack del modo abort, y llama al handler C con `fault_addr` en `r0`.
3. El handler C (`isolate_faulting_process`) emite la traza de entrada, guarda el pid actual, registra `fault_type` y `termination_reason = REASON_FAULT` en `pcb_ext_table[pid]`, y marca la tarea como `TERMINATED`.
4. `schedule_next_process()` selecciona la siguiente tarea `READY` (round-robin, saltando tareas `TERMINATED`). Si no existe ninguna tarea `READY`, el kernel entra en la política `idle/halt` documentada.
5. El handler C emite la traza de recuperación con el nuevo `current_pid`. El epílogo en ensamblador saca el stack del modo abort, cambia al modo `IRQ`, y salta a `context_restore`.
6. `context_restore` restaura el frame completo de registros de la tarea seleccionada desde su PCB y ejecuta `subs pc, lr, #0` (retorno de excepción a `USR`).

5. Trazas Requeridas

```
@ Al detectar el fallo:
MODE_SWITCH USER_TO_KERNEL pid=<n> reason=fault type=<type>

@ Tras el aislamiento y la reprogramación:
MODE_SWITCH KERNEL_TO_USER pid=<m> reason=fault_recovery
```

- `<n>` es el pid de la tarea que falló; `<m>` es el pid de la tarea recién planificada.
- `<type>` es uno de: `data_abort`, `prefetch_abort` (ver Sección 3).
- `<m> != <n>` en todos los casos manejados por esta política (la tarea que falló queda `TERMINATED`).

6. Estado del PCB Después de un Fallo

Campo del PCB	Valor después del fallo
<code>pcb_table[n].state</code>	<code>TERMINATED</code>
<code>pcb_ext_table[n].termination_reason</code>	<code>REASON_FAULT</code> (2)
<code>pcb_ext_table[n].fault_type</code>	<code>FAULT_TYPE_DATA_ABORT</code> (1) o <code>FAULT_TYPE_PREFETCH_ABORT</code> (2)
<code>pcb_ext_table[n].exit_code</code>	0 (no aplica para terminación por fallo)

7. Política de Idle / Halt

Si `schedule_next_process()` itera por todas las `MAX_PROCESSES` tareas y no encuentra ninguna en estado `READY`, el kernel ejecuta:

```
os_write("[KERNEL] Todos los procesos terminaron. Sistema en halt.\n");
while (1);    // Spin idle documentado - sin retorno
```

Esta política aplica tanto después de una terminación por fallo como después de `SYS_EXIT` normal cuando todas las tareas han finalizado. El kernel no intenta reiniciarse ni relanzar tareas.

8. Desviaciones y Limitaciones Conocidas

- DFSR/IFSR no decodificados: el subtipo de fallo (permiso, alineación, traducción) no se distingue en la Fase 2. Todos los data aborts emiten `type=data_abort` sin importar el estado de fallo del CP15.
- Sin MPU configurada: la protección de memoria se basa en la ausencia de mapeos a direcciones inválidas/periféricos, no en la aplicación de regiones por hardware. El mecanismo de contención es reactivo (abort al acceso) en lugar de proactivo.
- Fallos por instrucción indefinida: tratados como errores fatales del kernel (`halt b .`). Las tareas de usuario que generen instrucciones indefinidas bloquearán el sistema. Esta es una limitación conocida de la Fase 2.

9. Criterios de Aceptación — Fallos e Aislamiento

Los siguientes criterios deben cumplirse para la aceptación de la implementación de manejo de fallos:

✓	Criterio
☐	Una tarea con acceso a dirección inválida (ej. puntero nulo) es terminada y el kernel continúa ejecutando.
☐	Una tarea con fetch de instrucción ilegal (ej. <code>0xDEADC0DE</code>) es terminada y el kernel continúa ejecutando.
☐	Las tareas par restantes continúan ejecutándose después del aislamiento de la tarea fallida.
☐	Las trazas <code>reason=fault</code> y <code>reason=fault_recovery</code> aparecen en pares en el log serial.
☐	El PCB de la tarea terminada registra el <code>fault_type</code> correcto y <code>termination_reason = REASON_FAULT (2)</code> .
☐	No se produce escalada de privilegios: el fallo no otorga a la tarea acceso a modo kernel.
☐	No hay corrupción de contexto tras interleavings repetidos de <code>IRQ + syscall + fault</code> .
☐	Si todas las tareas terminan (por fallo o por exit normal), el kernel entra en la política de <code>halt</code> correctamente.